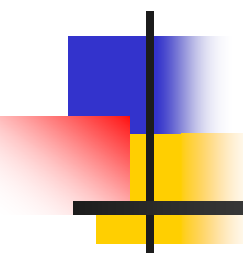


内部クラスとGUI

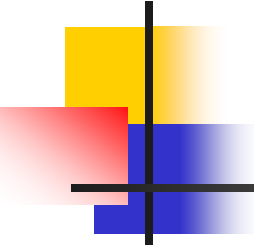
プログラミング第2同演習B

2026年6月26日

高田真吾

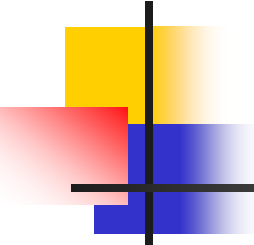


内部クラス



入れ子クラス (nested class)

- クラス定義の中で定義されたクラスのこと
 - 内部クラス (inner class)
 - static な入れ子クラス
 - ローカルな内部クラス
 - その派生として無名クラスというのもある
- 変数やメソッド定義と同じレベル.
- メソッド内で定義
- 知らなくてもプログラミング可能
 - でも, GUI プログラミングなどでときどき出てくるので, 読めるようになる必要はある.



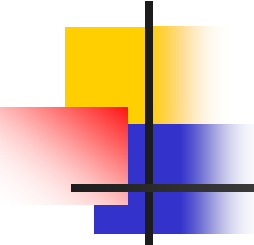
内部クラスの例

```
class InnerEx {  
    // AnInnerClassという内部クラスの定義  
    class AnInnerClass { int x=3; }
```

```
    public void doIt() {  
        AnInnerClass anInner = new AnInnerClass();  
        int result = anInner.x;  
        System.out.println(result);  
    }
```

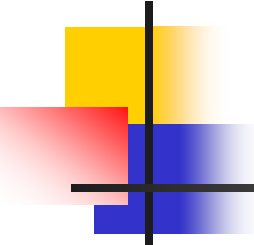
AnInnerClassのインスタンスを生成し、その中のxの値を得て、その値を出力している.

```
    public static void main (String[] args) {  
        InnerEx app = new InnerEx();  
        app.doIt();  
    }  
}
```



内部クラス

- 内部クラス(のインスタンス)が、外側のクラス(のインスタンス)と非常に密接な関係がある場合に用いる。
 - GUIを作成するときによく使う。
 - 内部クラスは、外側のクラスに定義された変数やメソッドを利用できる。
 - 例) InnerEx2クラス(次のスライド)
 - 内部クラスの正式名は「外側クラス名. 内部クラス名」である。
 - 例) InnerEx.AnInnerClass
 - InnerExをコンパイルすると:
 - InnerEx.class
 - InnerEx\$AnInnerClass.class
- の二つのファイルができる。



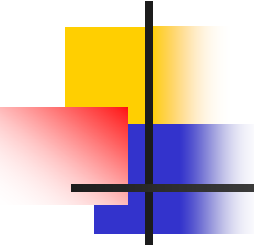
例: InnerEx2クラス

```
class InnerEx2 {  
    int x;
```

```
    class AnInnerClass {  
        AnInnerClass() {  
            x=3;  
        }  
    }  
}
```

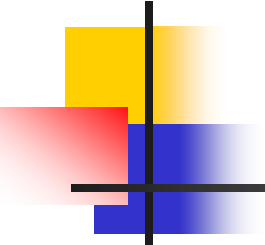
```
    public void doIt() {  
        AnInnerClass anInner  
            = new AnInnerClass();  
        int result = x;  
        System.out.println(result);  
    }
```

```
    public static void main(String[] args) {  
        InnerEx2 app = new InnerEx2();  
        app.doIt();  
    }  
}
```



修飾子

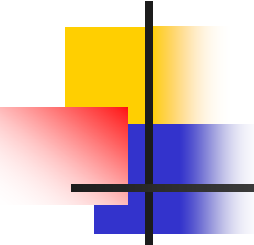
- 内部クラスで可以使用できる修飾子
 - public, private, protected
 - abstract
 - final
 - extends (継承)
 - implements (インタフェース)
- 注: 「static」を用いると, 内部クラスではなくなる.
 - 次の「staticな入れ子クラス」になる.



static な入れ子クラス

- 内部クラスと似ているが, "static" になる
 - クラスメソッドとインスタンス変数・メソッドの関係と同じ
 - 言い換えると・・・
 - 外側のクラスのインスタンス変数やインスタンスメソッドは参照不可
- 例:
 - ```
class InnerClass {
 int x;
 static class AnInnerClass {
 void method() {
 x = 3; // エラー
 }
 }
 ...
}
```



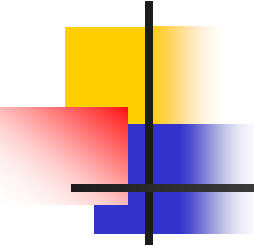


# ローカル内部クラス

- メソッドやコンストラクタの中で定義したクラス
  - クラスが特定のメソッドにおいてのみ使用されるのであれば、内部クラスではなく、ローカル内部クラスとしてできる。
  - クラス名がない場合は次の「無名クラス; anonymous class」
- 「ローカル」なので、ローカル変数と同様に：
  - public, private, staticなどの修飾子は駄目。

■ 例:

```
public void doIt() {
 class AnInnerClass {
 int x;
 int getX() { return x; }
 }
 AnInnerClass anInner = new AnInnerClass();
 System.out.println(anInner.getX());
}
```



# 無名クラス

---

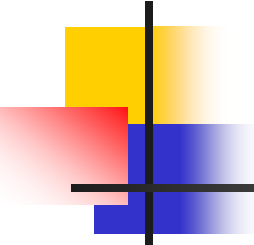
- 名前のないクラス

- クラスに名前がないので, new するときにクラスを定義する
  - 次のように書く  
new 親クラス名 () { 無名クラスの定義 }

- 例:

```
// Object クラスをスーパークラスとした無名クラス
public void doIt() {
 int result = (new Object() {
 int x;
 int getX() { return x; }
 }).getX();
 System.out.println(result);
}
```

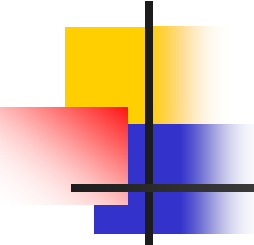




# GUI (Graphical User Interface)

---

- Javaは, GUIを作成するための部品をいろいろと提供している.
  - AWT, Swing, Applet, JavaFXなど
  - 注:他にもSWTなどthird-party部品が多数あるが,ここではjdk/jreが提供するものに限る.

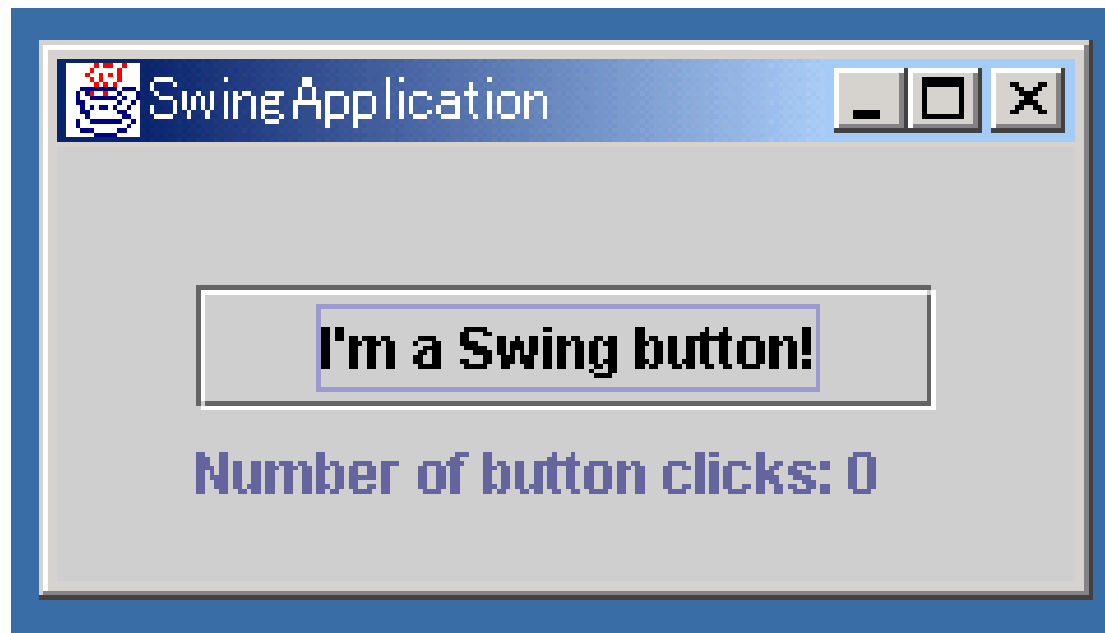


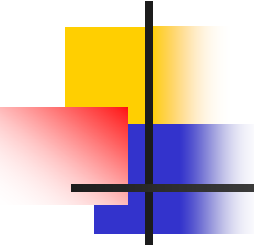
# AWT, Swing, JavaFX

---

- AWT (Abstract Window Toolkit)
  - Javaが提供している最も基本的なGUI部品
  - プラットフォームと密接に関係がある
    - Windows, Mac, Linuxで振る舞いが変わるかもしれない.
  - パッケージ名は, `java.awt` で始まる.
- Swing
  - AWTよりも多くのGUI部品を提供. → いろいろできる.
  - Javaで書かれている.
    - Look and feelを定義できる.
  - パッケージ名は, `javax.swing` で始まる.
- JavaFX
  - Java8からいったん標準となり, JDK11から別になった(OpenJFX)
  - Swingにも負けず, いろいろとできる.
  - パッケージ名は, `javafx`で始まる.

# 例1 : SwingApplication





# SwingApplication

---

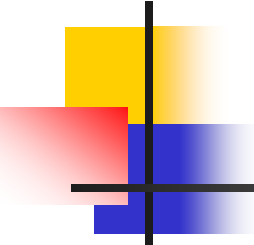
```
public class SwingApplication {
 private static String labelPrefix = "Number of button clicks: ";
 private int numClicks = 0;
 private JLabel label;
```

```
 class ButtonAction implements ActionListener {
 ...
 }
```

内部クラス  
(ボタンが押されたときに  
行う動作を定義)

```
 public Component createComponents() {
 ...
 }
```

```
 public static void main(String[] args) {
 ...
 }
}
```



# mainメソッド

```
public static void main(String[] args) {
```

```
 JFrame frame = new JFrame("SwingApplication");
 SwingApplication app = new SwingApplication();
 Component contents = app.createComponents();
 frame.getContentPane().add(contents, BorderLayout.CENTER);
```

```
 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 frame.pack();
 frame.setVisible(true);
```

```
}
```

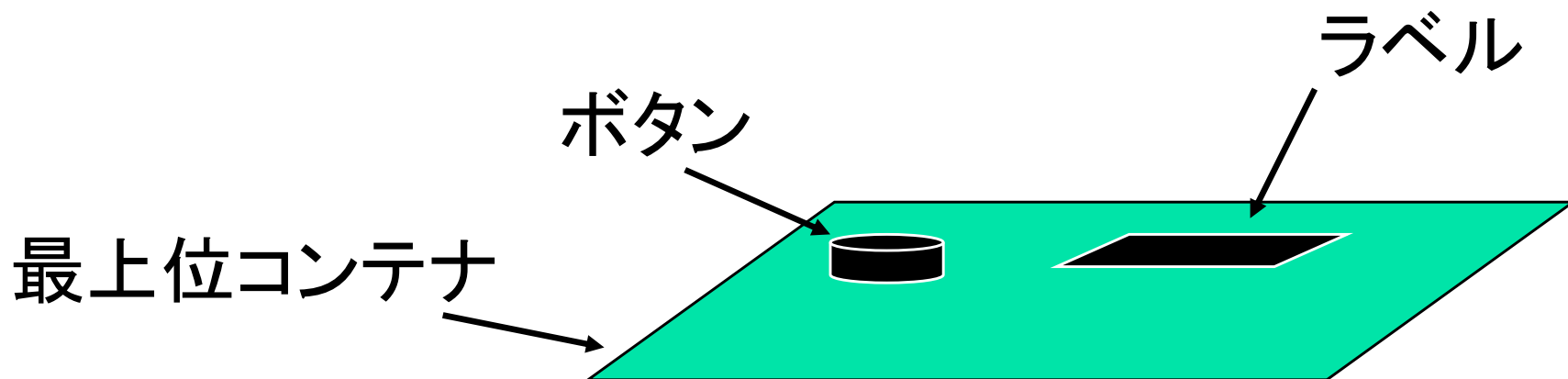
GUIの種々の  
部品の生成と  
配置

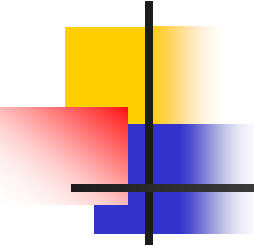
GUIの部品  
の設定  
(可視化等)



# 最上位のコンテナの設定

- 各部品間に親子関係を指定する.
- 最上位コンテナは一番上の親のこと.
- 最上位コンテナ: JFrame (or JDialog or JApplet)
- `JFrame frame = new JFrame (“SwingApplication”);`
  - JFrameクラスのインスタンスを作成する.
  - SwingApplication → ウィンドウのタイトルバーに表示する.





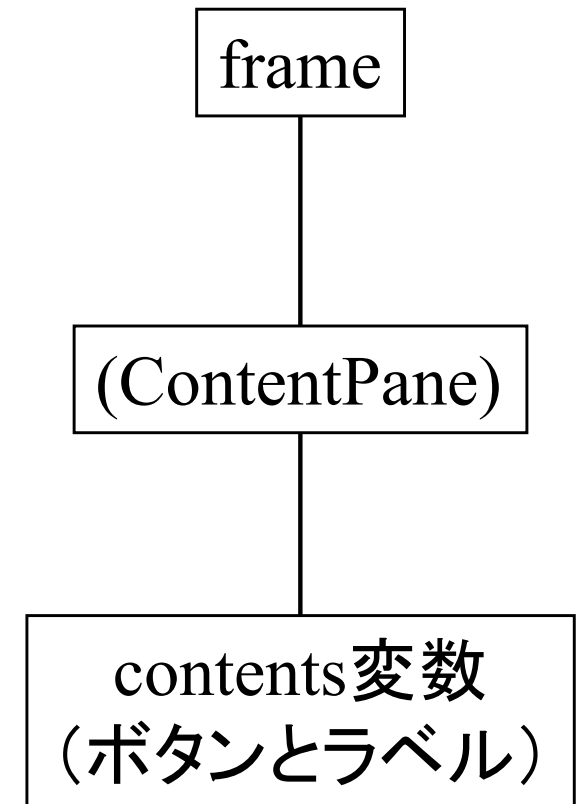
## mainメソッドの続き...

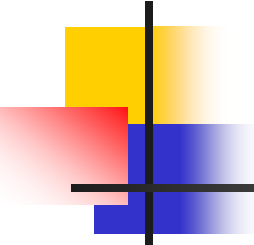
---

- インスタンスの生成
  - `SwingApplication app = new SwingApplication( );`
- 様々な部品の作成
  - `Component contents = app.createComponents( );`
  - `contents`
    - ボタンとラベルをまとめたもの
  - 詳細は後で

# 最上位コンテナ内への部品の配置

- `frame.getContentPane().add(contents, BorderLayout.CENTER);`
- `frame`(最上位コンテナ)に`contents`(ボタンとラベル)を追加
  - 厳密には, `frame`の下に`ContentPane`が存在
  - UIの各種部品は`ContentPane`の子(または子孫)となる。(メニューバーは例外)
- 配置の仕方は「中央」(CENTER)
  - 左に寄せたりできるが, この例では`contents`でまとめてある一つの部品しかないため, 意味がない。
  - 他に, NORTH, SOUTH, EAST, WESTなどがある。(詳細はAPIドキュメントでBorderLayoutを見ましょう!)





# GUI部品の設定

---

- `frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);`
  - 「ウィンドウを閉じた」というイベントが発生したときにアプリケーションが正常に終了するため.
- `frame.pack();`
  - `frame`の子供の部品の推奨サイズに基づいて自動的に`frame`の大きさを計算し, 設定する.
  - 明示的に設定することもできる.  
(例) `frame.setSize(500, 600);`
- `frame.setVisible(true);`
  - 作成した`frame`を見えるようにする.
  - `new JFrame()`だけでは見えない!!!

# 様々な部品の作成

Component contents = app.createComponents();

```
public Component createComponents() {
```

```
 label = new JLabel(labelPrefix + "0 ");
 JButton button = new JButton("I'm a Swing button!");
```

```
 ButtonAction buttonListener = new ButtonAction();
 button.addActionListener(buttonListener);
```

```
 JPanel pane = new JPanel();
 pane.setBorder(BorderFactory.createEmptyBorder(30, 30, 10, 30));
 pane.setLayout(new GridLayout(0, 1));
 pane.add(button);
 pane.add(label);
```

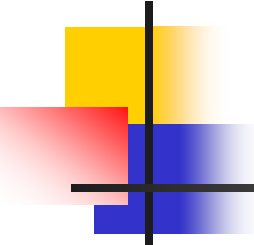
```
 return pane;
```

```
}
```

ラベルとボタンの  
生成

ボタンが押された  
ときの動作の設定

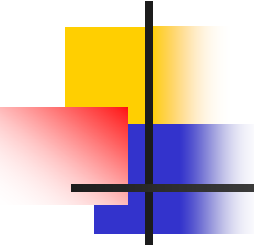
ラベルとボタンを一つにまとめる。  
同時にボーダーを付加。



# ラベルとボタンの生成

---

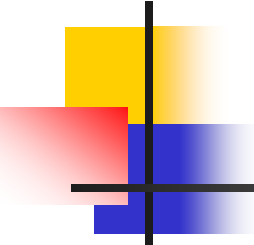
- `label = new JLabel(labelPrefix + “0 ”);`
  - JLabelのインスタンスの生成
  - コンストラクタの引数値は, ラベルそのもの
  - labelはインスタンス変数
- `JButton button =  
new JButton(“I’m a Swing button!”);`
  - JButtonのインスタンスの生成
  - コンストラクタの引数値は, ボタンのラベル
  - buttonはローカル変数



# ボタンが押されたときの動作の設定

```
ButtonAction buttonListener = new ButtonAction();
button.addActionListener(buttonListener);
```

- ButtonActionクラスは内部クラス
  - 次のスライド
- そのインスタンスを、作成しているボタンのActionListenerとして登録する.
  - ActionListenerそのものはインタフェース
  - 何らかの「アクションイベント」が発生したら、自動的にactionPerformed()メソッドが呼び出される.
  - 「アクションイベント」→ ボタンを押した



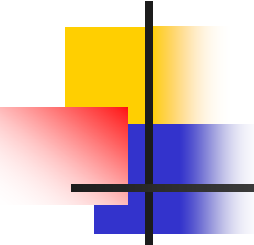
# ButtonActionクラス

---

- ボタンを押すと、「ボタンが押された」というイベントが発生する.
- actionPerformed()が自動的に呼び出される.
- numClicksの値が1増える.
- ラベルで表示されている文字列を変更する.

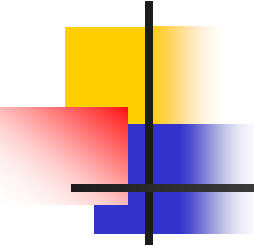
```
class ButtonAction implements ActionListener {
 public void actionPerformed(ActionEvent e) {
 numClicks++;
 label.setText(labelPrefix + numClicks);
 }
}
```





## ボタンの動作の設定：無名クラスの場合

- ButtonActionクラスのようにメソッドを一つしか定義していない場合，無名クラスを使うこともできる.
  - ```
ActionListener buttonListener = new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        numClicks++;  
        label.setText(labelPrefix + numClicks);  
    }  
};  
button.addActionListener( buttonListener );
```
- 2文目のbuttonListenerを1文目の右辺に置き換えることもできる.
- どちらにするかは好みの問題.

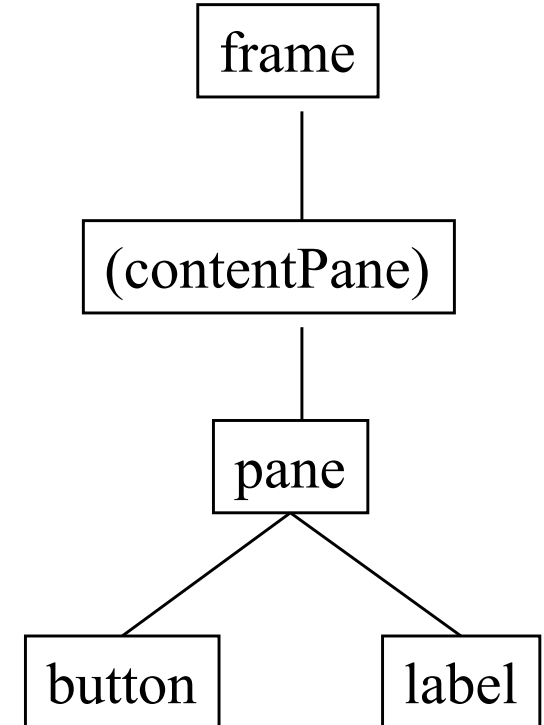


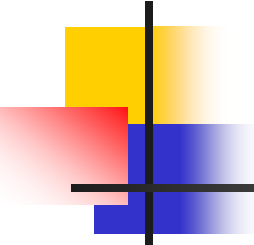
ラベルとボタンをまとめる (1)

- `JPanel pane = new JPanel();`
 - JPanelのインスタンスの生成
 - JPanelは, いくつかのGUI部品を部分的にまとめておきたいときに用いる.
- `pane.setBorder(BorderFactory.
createEmptyBorder(30, 30, 10, 30));`
 - paneの端に何もない(empty)境界線(border)を置く.
 - (30, 30, 10, 30)は, それぞれpaneの上, 左, 下, 右の端を指す. (単位はピクセル)

ラベルとボタンをまとめる (2)

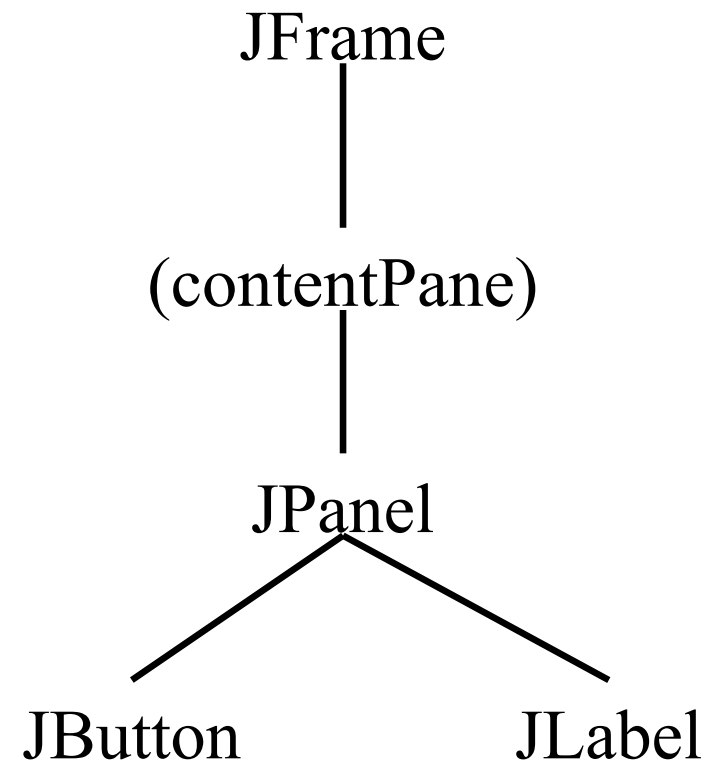
- `pane.setLayout(new GridLayout(0, 1));`
 - `pane`内の部品の配置を決めるための「レイアウトマネージャ」の設定
 - `GridLayout(x, y) → x行y列`
 - `x`または`y`が0 → 横一行から縦一列
 - 今は縦一列
 - 他に`BoxLayout`などがある.
- `pane.add(button);`
`pane.add(label);`
 - ボタンとラベルを`pane`に追加する.
 - 大きさや位置は前の行で設定した方法で決まる.

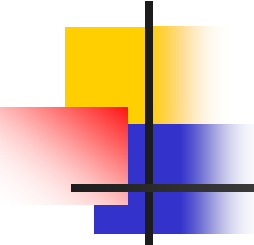




まとめ:コンテナと部品階層

- 最上位コンテナ → JFrame, JDialog, JApplet
- 最上位コンテナの子供にContentPaneがある
- 途中状態にも部分的に
まとめるコンテナを入れる
ことも可能
 - JPanel
 - JScrollPane
 - JTabbedPaneなど





おわりに

- 本日はSwingの基礎を扱った.
- Swingにはいろいろな部品があり, 本日の内容を応用すればプログラミングは(一応)できる.